

Return type deduction and SFINAE

1. Introduction

Currently, a failure to deduce function's return type results in a hard error. In this paper we analyze the impact of this behavior on the existing language; in particular, how it compromises the goals of Concepts Lite: descriptive error messages and concept-based overloading.

Secondly, we analyse possible ways of addressing the problems, with the recommendation of introducing a special "expression-body" syntax for declaring function template bodies.

The proposed syntax, also allows generic lambdas to be used with concept-constrained libraries (like Ranges TS), without compromising their major features (better error messages, overloading) while preserving their terseness:

Problematic statement	Current workaround	Proposed solution
<pre>std::vector<std::strings> strs{ /*...*/ }; ranges::find_if(strs, [](auto const& s) { return s == 2; });</pre>	<pre>ranges::find_if(strs, [](auto const& s) -> decltype(s == 2) { return s == 2; });</pre>	<pre>ranges::find_if(strs, [](auto const& s) => s == 2));</pre>
Problem above: return type deduction in the lambda prevents compiler from performing concept checks and generating a short and readable error message (see section 4.2.1 for details).		
<pre>void on_each_month(day_point start_date, int count, Invokable<day_point> f); void on_each_month(day_point start_date, int count, Invokable<std::string_view> f); on_each_month(day_point{2017_y/10_m/20_d}, 10, [](auto d) { return day_of_week(d) == friday; });</pre>	<pre>on_each_month(day_point{2017_y/10_m/20_d}, 10, [](auto d) -> decltype(day_of_week(d) == friday) { return day_of_week(d) == friday; });</pre>	<pre>on_each_month(day_point{2017_y/10_m/20_d}, 10, [](auto d) => day_of_week(d) == friday);</pre>
Problem: return type deduction prevents the compiler from performing concept checks and selecting day_point overload, which makes the code ill-formed instead (see section 4.2.2 for details).		
<pre>struct Employee { std::string const& first_name() const; std::string const& last_name() const; }; std::vector<Employee> ve{ /*...*/ }; ranges::sort(ve, std::less<>(), [](Employee const& e) { return e.first_name(); });</pre>	<pre>ranges::sort(ve, std::less<>(), [](Employee const& e) -> decltype(auto) { return e.first_name(); });</pre>	<pre>ranges::sort(ve, std::less<>(), [](Employee const& e) => e.first_name());</pre>
Problem: lambda by default returns by value, this creates unnecessary copies of std::string objects (two copies for each performed comparison). Use of decltype-based return type eliminates the problem, and will remain correct if function is changed to return std::string_view by value (see section 6.1 for details).		

2. Revision history

2.1. Revision 1

This revision of the paper extends the motivation section to better illustrate the impact on the Concept TS features. In addition, the discussion section was extended to include the discussion of alternative solutions for the problem.

Finally, we no longer propose to treat *any* return type deduction failure as a SFINAE-able error. Instead, we propose to narrow the solution to certain set of cases, explicitly designated by the use of expression body syntax: => expr that was proposed in [P0573R0: Abbreviated Lambdas for Fun and Profit](#).

3. Motivation and Scope

Let's consider the following implementation of `variadic_negator` — an adapter that takes a predicate and returns the opposite value to the one returned by the original:

```
template<typename F>
class variadic_negator_functor
{
    F f_;

public:
    explicit variadic_negator_functor(F a_f) : f_(a_f) {}

    template<typename... Args>
    auto operator()(Args&&... args)
        -> decltype(!this->f_(std::forward<Args>(args)...))
    { return !this->f_(std::forward<Args>(args)...); }
};

template<typename F>
variadic_negator_functor<F> variadic_negator(F f)
{
    return variadic_negator_functor<F>(std::move(f));
}
```

In the implementation, we can see an obvious repetition of the function invocation that is placed both in the return type declaration and in the return statement. We could think of avoiding this repetition by using function's return type deduction and declare the return type as `decltype(auto)`. But the resulting code would not have the same semantics as before. As a consequence of this change we would affect the overload resolution for calls to `operator()`: when we used `decltype(expr)` this candidate was not considered viable in cases when `f` was not callable with `args...`, or the result of the invocation could not be negated. In contrast, when we use `decltype(auto)` this candidate is always viable.

When there is only one `f` to be considered as candidate to call `variadic_negator(f)(args...)`, the nuanced difference explained above might seem irrelevant, as it will only change the error messages caused by invalid invocations of the functor from "no viable candidate found" to "(long) error instantiatin a template". However, the negative results are more grave when the viability/validity of other functions depends on the validity of the call to `variadic_negator`. For example, consider the following:

```
template<typename F>
auto invoke(F f) -> decltype(f(0)); //1

template<typename F>
auto invoke(F f) -> decltype(f(std::string())); //2

bool test(int);
```

Given the above declarations the expression `invoke(&test)` is valid and selects the first overload, the same applies to `invoke(variadic_negator(&test))` if `decltype(expr)` is used for return type. However, in case of `decltype(auto)` return type, the code will emit hard error about the lack of conversion from `std::string` to `int` while instantiating a function template.

To understand this situation we need to realize that expression in the form `decltype(foo(args...))` requires compiler to determine return type of the selected overload of `foo` which in case of function template that uses return type deduction leads to its instantiation, and according to the current wording, any error occurring in such instantiation is considered to be a hard-error. In case of our example the expression `invoke(variadic_negator(&test))` leads to the instantiation of `operator()` with `std::string&`, and that is invalid as `&test` accepts only integers.

3.1. Functions accepting std::function as parameter

It may be pointed out that the presented example of the `invoke` function is artificial and such situations will not normally occur in the code. However, as the result of resolution of the [LWG issue #2132](#) a similar mechanism is used to eliminate `std::function` constructor in case when the object is not callable. As a consequence, the same problem would occur in case `invoke` was defined as follows:

```
bool invoke(std::function<bool(int)> f);           //1
bool invoke(std::function<bool(std::string)> f); //2
```

3.2. Interoperation with lambda functions

The problem described by this paper occurs only in situations when a functor with templated `operator()` that uses return type deduction is passed to a function. As writing such classes is rarely involved in day-to-day programming, its impact may be neglected as affecting only experts, that should be aware of above SFINAE nuances and design their code accordingly.

However, with generic lambdas in C++14 we have introduced short syntax for declaring such classes; for example expression `[](auto x) { return !test(x); }` creates an unnamed class with template `operator()` that uses `auto` return type deduction. As a consequence programmers are exposed to the quirks of the detailed behavior of return type deduction even in simple code, like lambda implementation of a negator.

If we consider the following declarations:

```
bool test(int);

bool invoke(std::function<bool(int)> f);           //1
bool invoke(std::function<bool(std::string)> f); //2
```

The call to `invoke` with `[](int x) { return !test(x); }` is well formed and selects the first overload, while seemingly identical expression that uses the new generic lambda `[](auto x) { return !test(x); }` produces a hard error.

Furthermore, it is worth noting that in the above example the created closure is not used in a polymorphic way and we could use an `int` parameter. However, it may be pointed that `auto` lambda parameters lead to better code, in the same way as in the use `auto` instead of specific type in variable declarations (no unintended conversion, better maintainability, etc...).

4. Impact on Concepts

In this section, we discuss the relation between SFINAE (and by consequence proposed feature) and currently proposed Concept TS design.

4.1. Concepts checks depends on SFINAE

One of major additions of the proposed Concepts design is the `requires` expression, which provides a simple syntax for checking the validity of a given expression performed on given types. However, the underlying mechanism is same as in existing expression-SFINAE implementations, and in order to work properly it requires that functions used in expression must be properly *constrained*: they should not be viable candidates unless they body is well formed. As a consequence, it is required that the operations tested in constrained function must also be constrained.

To illustrate the above point, let's consider the following (very simplified) concept hierarchy and constrained generic function overloads:

```
template<typename I>
concept bool ForwardIterator = requires (I i) { ++i; };

template<typename I>
concept bool BidirectionalIterator = ForwardIterator<I> && requires (I i) { --i; };

template<typename I>
    requires ForwardIterator<I>
I rotate(I first, I last) {
    /* fake body that uses provided operations */
    ++first; return first;
}

template<typename I>
    requires BidirectionalIterator<I>
I rotate(I first, I last) {
    /* fake body that uses provided operations */
    ++first; --first; return first;
}
```

Now, consider the following types modelling the above concepts:

```
struct Iter {
    Iter operator++() { return *this; };
};

template<typename I>
class FixedReturnWrapper
{
    I pos;

public:
    FixedReturnWrapper(I s) : pos(s) {}

    FixedReturnWrapper operator++() { return FixedReturnWrapper(++pos); }
    FixedReturnWrapper operator--() { return FixedReturnWrapper(--pos); }
};

template<typename I>
class AutoReturnWrapper
{
    I pos;

public:
    AutoReturnWrapper(I s) : pos(s) {}

    auto operator++() { return AutoReturnWrapper(++pos); }
    auto operator--() { return AutoReturnWrapper(--pos); }
};

template<typename I>
class DecltypeReturnWrapper
{
public:
    I pos;

    DecltypeReturnWrapper(I s) : pos(s) {}
};

template<typename I>
auto operator++(DecltypeReturnWrapper<I>& d)
    -> decltype(DecltypeReturnWrapper<I>{++d.pos}) { return DecltypeReturnWrapper<I>{++d.pos}; }

template<typename I>
auto operator--(DecltypeReturnWrapper<I>& d)
    -> decltype(DecltypeReturnWrapper<I>{--d.pos}) { return DecltypeReturnWrapper<I>{--d.pos}; }

template<typename I>
class ConceptWrapper
{
    I pos;

public:
    ConceptWrapper(I s) : pos(s) {}

    auto operator++() requires ForwardIterator<I> { return ConceptWrapper(++pos); }
    auto operator--() requires BidirectionalIterator<I> { return ConceptWrapper(--pos); }
};
```

Finally, let's consider the effects of the following calls ([live example](#)):

```
rotate(Iter{}, Iter{}); //1
rotate(FixedReturnWrapper{Iter{}}, FixedReturnWrapper{Iter{}}); //2
rotate(AutoReturnWrapper{Iter{}}{}, AutoReturnWrapper{Iter{}}); //3
rotate(DecltypeReturnWrapper{Iter{}}, DecltypeReturnWrapper{Iter{}}); //4
rotate(ConceptWrapper{Iter{}}, ConceptWrapper{Iter{}}); //5
```

Invocations with types `Iter` (1), `DecltypeReturnWrapper<Iter>` (4) and `ConceptWrapper<Iter>` (5) are calling the `rotate` overload that is accepting the `ForwardIterators`. This is a desired and expected behaviour, as operations on these types are properly *constrained*: candidate functions are viable candidates *only* when they body is well formed. This is achieved by either by being a non-template function or by the usage of expression SFINAE (possibly in the form of a `requires` clause).

Regretfully, in the case of the `FixedReturnWrapper<Iter>` (2) we encounter a compilation error from body of `FixedReturnWrapper::operator--`, which is called from `rotate` function. This is caused by the fact that `operator--` for this type is effectively *underconstrained*: it is considered to be a viable candidate even if its body is ill formed, which in consequence leads to a false-positive result of `Bidirectional<FixedReturnWrapper<Iter>` check and the selection of the wrong overload of function `rotate`.

Finally, in case of the `AutoReturnWrapper<Iter>` (3) we encounter a compilation error from the body of the `FixedReturnWrapper::operator--` instantiated while performing return type deduction for the `auto` placeholder. As any other error occurring during function template body instantiation, it is treated as hard error; this basically stops the compilation, thus limiting the ability to select the appropriate overload of function `rotate` (the `ForwardIterator` version).

As we can see from the example of the `FixedReturnWrapper<Iter>` (2) and `AutoReturnWrapper<Iter>` (3), concepts fails to deliver their major promises: more readable error messages and overloading based on the subsumption in situation when they are fed with an unconstrained argument.

4.2. Lambdas as arguments for constrained parameters

The introduction of C++14's generic lambdas provides an easy way to create unconstrained functors, which will be prone to problems described in the previous section, when they are used with constrained functions. At the same time one of the major usages for the lambda-expression is to provide predicates/functors for the STL algorithms, which was selected as flagship conceptualized library. In addition, due to the ad-hoc nature of the lambdas, the possibility that the Standard Library will already contain matching concepts (like in the `Iterator` case) is very limited; which leaves users with only a few workarounds, like: creating their own concepts (which in contrast to lambdas cannot be local to function), or using expression SFINAE or ad-hoc `requires` clauses (which introduces code repetition).

4.2.1. Error messages

To illustrate the problem with error messages, let's consider following simplified version of `find_if` (we focus only on the functor constrain):

```
template<typename F, typename... T>
concept bool Predicate = requires(F f, T... t) {
    { f(t...) } -> bool;
};

template<typename Iterator, Predicate<typename std::iterator_traits<Iterator>::reference> F>
Iterator find_if(Iterator b, Iterator e, F f);
```

and the following invocations ([live example](#)):

```
std::vector<std::string> vs;
find_if(vs.begin(), vs.end(), [](auto const& s) { return s == 2; }); //1
find_if(vs.begin(), vs.end(), [](auto const& s) -> decltype(s == 2) { return s == 2; }); //2
```

In the case of the second invocation (2), the compiler will produce a short error message indicating that the suplied lambda does not model concept `Predicate`:

```
prog.cc:11:10: note:   constraints not satisfied
    Iterator find_if(Iterator b, Iterator e, F f);
    ~~~~~
prog.cc:6:14: note:   within 'template<class F, class ... T> concept const bool Predicate<F, T ...> [with F = main()::<lambda(const auto:1&)>; T = {std::__iterator_traits<__gnu_cxx::__normal_iterator<std::string const&*>, __gnu_cxx::__normal_iterator<std::string const&*>*>}>]'
    concept bool Predicate = requires(F f, T... t) {
    ~~~~~
prog.cc:6:14: note:   with 'main()::<lambda(const auto:1&)>' f'
prog.cc:6:14: note:   with 'std::__cxx11::basic_string<char*> t#0'
prog.cc:6:14: note:   the required expression 'f(t ...)' would be ill-formed
```

However, in case of the first invocation (1), the compiler produces a few hundreds of lines of errors, mentoining the lack of comparision operator between `std::string` and `int`. This is caused by the fact that the concept check for parameter `F` of `find_if` (performed in both examples) requires the determination of the return type for the supplied functor with `std::string&` argument. In case of the first example, this triggers return type deduction for the lambda, and in consequence leads to its instatation with provided arguments — as the error produced from this instatiation is a hard error, compiler emits error from the instation, and stops processing in the middle of the concept check.

4.2.2. Overloading using concepts

Let's consider the case of the following `on_each_month` functions:

```
void on_each_month(day_point start_date, int count, Invokable<day_point> f);
void on_each_month(day_point start_date, int count, Invokable<std::string_view> f);
```

The goal of this function is to call the provided functor for the specified `start_date`, and then same day next month, two months in the future, up to `count - 1` months in future. The first overload provides a `day_point` in the numeric format (we assume very similar format to [Howard Hinnant's date library](#)). The second is an optimized version that feeds provided function with the serialized date — by providing this overload we can avoid performing multiple allocations and perform operation directly on this representation (incrementing month).

Now, consider the following invocation, where `day_of_week` is a library function returning a duration since midnight:

```
on_each_month(day_point{2017_y/10_m/20_d}, 10, [](auto d) { return day_of_week(d) == friday; });
```

This code will fail to compile, due to an error in the return type deduction for the invocation of the lambda with the object of type `std::string_view`, caused by checking the viability of the second overload of `on_each_month`. To fix above problem the user is required to either:

- change the lambda parameter type to `day_point`
- constrain the lambda with whatever function `day_of_week` is constrained with
- infer constrains from `day_of_week` by specifying the lambda's return type: `[](auto d) -> decltype(day_of_week(d) == friday) { return day_of_week(d) == friday; }`

Each of the above solutions introduces code repetition: either via repeating entire lambda body, or reproducing the constrains of the library function; or makes the code less robust by committing to the specific resolution of time (using fixed `time_point` type).

5. Alternative solutions

In this section we examine possible solutions for *constraining* function templates, discussing their viability and impact both on the future of the language and on user programs.

5.1. Constraining functions using Concepts

The most obvious solution is to assume (and, in consequence, require) that all operations on objects used with the argument, will be constrained via appropriate concepts.

First, let's consider the already presented example of class `variadic_negator`, defined as follows:

```
template<typename F>
class variadic_negator_functor
{
    F f_;

public:
    explicit variadic_negator_functor(F a_f) : f_(a_f) {}

    template<typename... Args>
    decltype(auto) operator()(Args&&... args)
    { return !this->f_(std::forward<Args>(args)...); }
};
```

At first glance we may consider that this function can be easily constrained by the standard concept `Predicate`:

```

template<typename F>
class variadic_negator_functor
{
    F f_;

public:
    explicit variadic_negator_functor(F a_f) : f_(a_f) {}

    template<typename... Args>
        requires Predicate<F&, Args...>
    decltype(auto) operator()(Args&&... args)
    { return !this->f_(std::forward<Args>(args)...); }
};

```

A closer look into this code exposes a flaw in this approach: by using concept `Predicate` we impose the restriction on the `F` return type, to be convertible to `bool`. In contrast, the original design supported any object on which `operator!` can be applied. To restore original functionality we may relax the constraint, by using concept `Invokable` instead:

```

template<typename F>
class variadic_negator_functor
{
    F f_;

public:
    explicit variadic_negator_functor(F a_f) : f_(a_f) {}

    template<typename... Args>
        requires Invokable<F&, Args...>
    decltype(auto) operator()(Args&&... args)
    { return !this->f_(std::forward<Args>(args)...); }
};

```

However, in this situation, the function again becomes *underconstrained*, as we no longer guarantee the call to `operator!` on the result of `f` will be well-formed.

Finally, to restore the original functionality of this simple class, we need to combine concept `Invokable<F&, Args...>` with an additional constrain, requiring result type being negatable. To achieve this we will need to add an ad-hoc expression constraint (by using `decltype(expr)` or an in-place `requires` expression), which will re-introduce the code repetition that we want to eliminate.

Alternatively, we may consider introducing a (set of) standardized syntactic constrains (like `has_negation`) to be used in such situation, but by doing so, we will essentially recreate a failed pre-C++11 concept design with its overwhelming number of concepts.

5.2. Turning *any* return type deduction failure into soft-error

In this section, we discuss the originally proposed solution, based on turning the return type deduction failures into soft errors. Choosing this solution will basically resolve all the problems related to uses of `auto` return type, by making them *constrained*: they, by definition, will be viable if and only if their body is well-formed.

However, it has its own drawbacks. Firstly it promotes defining the interface of the function in terms of terms of its implementation. This should be considered a bad practice for the large function, like generic algorithms. For example, we may consider the implementation of the sort algorithm, that will be based on the quick-sort, which performs only swap on the elements. Inferring interface (requirement on accepted iterators and their value types) from this implementation, will prevent future optimization, like switching to insertion sort after a certain number of recursive calls, as they will require additional operations on the types (creating an copy), and will break existing calls.

Secondly, this approach effectively requires compilers to be able to backtrack from any error occurring in the function body. As a consequence, newly introduced language features would need to be considered in this context. This will increase the amount of work required to introduce any new language features, like `constexpr-if` or structured binding, that are currently limited to function bodies and cannot appear in unevaluated context. In the end such solution would cripple the evolution of the language.

Finally, the complexity of the implementation of such feature, suggest an alternative solution that will reduce SFINAE-able errors to only certain constructs in function bodies. One of the options, is to limit this capability only to function with a body consisting of a single return statement. The author believes that such design will be very surprising for the user, because even a simple refactoring, like splitting large expression and saving their result in local variables, will cause previously well-formed code to emit compilation error from a seemingly unrelated template instantiation. For reference, please check the difference between invocation of (4) and (2) from [4.1. Concepts checks depends on SFINAE](#) section.

6. Proposed solution: expression body

In this chapter, we present the recommended solution for the problem of constraining small functions, based on the introduction of the new function definition syntax, limited to functions consisting of a single expression. We propose to reuse the syntax proposed in [P0573R0: Abbreviated Lambdas for Fun and Profit](#), and making `=> expr` syntax equivalent to `noexcept(noexcept(expr)) -> decltype(expr)` { `return expr;` }, while extending it to be allowed for all functions (including member functions).

To illustrate, the following definition:

```

template<typename C> auto size(C const& c) => c.size();

```

will have same semantics as:

```

template<typename C> auto size(C const& c) noexcept(noexcept(c.size())) -> decltype((c.size())) { return c.size(); }

```

It allows to infer the interface of the function (including SFINAE validity) from its implementation, but still can be combined with a proper concept constraint; for instance, in case of the `variadic_negator_example`, we can still express requirement of callability via a concept (and benefit from better error messages), without losing the flexibility and correctness of original solution:

```

template<typename F>
class variadic_negator_functor
{
    F f_;

public:
    explicit variadic_negator_functor(F a_f) : f_(a_f) {}

    template<typename... Args>
        requires Invokable<F&, Args...>
    auto operator()(Args&&... args) => !this->f_(std::forward<Args>(args)...);
};

```

This feature also enables a practice for constraining templates indicated in [6], that apart from decent concepts will well defined axioms and semantics you sometimes need a simpler, pure syntactic constraint. The expression body allows such limited syntactic only constraints for ad-hoc use, while leaving full intuitive, axiomatized constraints for concepts.

Also, limiting the function body to a single expression is actually beneficial, as it prevents overuse of the facility for larger functions (like the discussed sort example), while still covering large amount of use cases.

6.1. Using lambda to write projections

The return type deduction for a lambda, that returns an object by copy (the `auto` deduction), is not well suited for writing projections (lambda returning a member, calling a getter). This will become more important in the case of the [Ranges TS](#) that allow projections to be provided to the standard algorithm.

For example, in the following code:

```

struct Employee
{
    std::string first_name;
    std::string last_name;
};
std::vector<Employee> ve{ /*...*/ };

```

The invocation of the form:

```

ranges::sort(ve, std::less<>(), [] (Employee const& e) { return e.first_name; });

```

will create two copies of string **for each** comparison of employee objects, creating $O(ve.size())$ unnecessary copies of the string in total.

In contrast, if we declare projection using the proposed expression body syntax:

```
ranges::sort(ve, std::less<>{}, [] (Employee const& e) => e.first_name);
```

the lambda will return a (const) reference to the string (note the use of double parentheses in `decltype((expr))`), and the code will not create any additional copies compared to an STL1-like solution:

```
ranges::sort(ve, [] (Employee const& e1, Employee const& e2) { return e1.first_name < e2.first_name; });
```

Furthermore, in case of predicates (and other lambdas producing a new object), result will be returned by value regardless of which syntax is used, so the following lambdas functions will have a `bool` return type:

```
[] (Employee const& e1, Employee const& e2) { return e1.first_name < e2.first_name; }  
>[] (Employee const& e1, Employee const& e2) => e1.first_name < e2.first_name
```

6.2. Inferring `noexcept`

We also require an expression body to infer a function exception specification from the returned expression. This matches recent development of the language, that makes [exception specification part of type system](#).

Furthermore, with the direction of terminating the program in case of an exception being thrown from an invocation of the functor passed to a parallel algorithm, we need the ability to check if a given function may throw an exception, to allow a seamless parallelization of user code:

```
template<typename It, typename F>  
void parallel_for(It first, It last, F f)  
{  
    if constexpr(noexcept(f(*first)))  
        std::for_each(std::par, first, last, f);  
    else  
        std::for_each(first, last, f);  
};
```

We need to avoid the use of `std::par` execution policy in case of a throwing functor, to prevent program from termination, in case of an exception.

Some concerns may be also raised that marking the lambda as `noexcept`, in generic case, requires the insertion of additional handling code that will `std::terminate`, which may negatively impact the performance of a program. However, in the case of the proposed functionality, lambda will be marked as `noexcept`, only if its body is know not to produce any exceptions (any eventual handling code is already present in code of called functions), so adding this specification should not impact the generated code.

7. Proposed wording

To be defined.

The general idea is to introduce new expression body syntax, that can be used for the normal functions, template functions, and lambda expressions. We propose that functions using expression body:

- will be implicitly treated as inline,
- need to use trailing return type introducer: `auto`,
- cannot be forward declared.

In case of the potential ambiguity caused by use of lambda with expression body as middle argument of the function, we propose to choose shortest possible expression as body, i.e.

```
f([](auto x) => x + 2, 3, 4);
```

should be interpreted as:

```
f([](auto x) => x + 2), 3, 4);
```

8. Implementability

Proposed expression body is basically a terse notation for existing language syntax, that avoids unnecessary code repetition and can be already simulated using following macro:

```
#define EXPR_BODY(expr) noexcept(expr) -> decltype((expr)) { return expr; }
```

Given above it should not impose major implementation problems for compilers already implementing expression-SFINAE.

9. Acknowledgements

Andrzej Krzemieński, Casey Carter and Ville Voutilainen offered many useful suggestions and corrections to the proposal.

10. References

1. Marshall Clow, "C++ Standard Library Defect Report List (Revision R93)" (N4485, <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/n4485.html>)
2. Howard Hinnant, "date library" (<https://github.com/HowardHinnant/date>)
3. Barry Revzin, "Abbreviated Lambdas for Fun and Profit" (P0573R0, <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0573r0.html>)
4. Eric Niebler, Casey Carter, "Working Draft, C++ Extensions for Ranges" (N4651, <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/n4651.pdf>)
5. Jens Maurer, "P0012R1: Make exception specifications be part of the type system, version 5" (P0012R1, <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/p0012r1.html>)
6. Andrew Sutton, Bjarne Stroustrup, "Design of Concept Libraries for C++" (<http://www.stroustrup.com/sle2011-concepts.pdf>)